

HelixLLM

HelixLLM

One binary, six modes — OpenAI- and Anthropic-compatible inference from your laptop to a multi-host cluster.

Summary

HelixLLM is an enterprise-grade distributed LLM system in Go: a single binary with a mode system that scales from single-host development to multi-host production. It serves fully OpenAI- and Anthropic-compatible APIs over HTTP/3, with local llama.cpp inference, a scored multi-provider fallback chain, a RAG pipeline, and a ReAct agent system.

Short description

HelixLLM is a single-binary, Go-based distributed LLM system. It exposes OpenAI- and Anthropic-compatible APIs over HTTP/3, runs local llama.cpp inference, auto-discovers and scores free cloud providers into a failover chain, and adds a RAG knowledge pipeline plus a tool-calling ReAct agent — deployable in six modes.

Long description

HelixLLM is an enterprise-grade distributed LLM system built in Go with Gin, and its central trick is that one artifact serves every scale. It compiles to a single binary whose mode system decides at deploy time what that binary *is*: run it as `full` for an all-in-one instance on a laptop, or split responsibilities across gateway, brain, knowledge, agents, and control modes spread over multiple hosts — the same code, re-arranged rather than rewritten, from a developer's machine to a production cluster.

It speaks two dialects fluently: fully OpenAI- and Anthropic-compatible APIs, so existing SDK clients from either ecosystem work unmodified, all served over HTTP/3 (QUIC) with automatic HTTP/2 fallback and TLS 1.3. Local inference runs via llama.cpp with CUDA, Metal, and ROCm support, so the same build accelerates on Nvidia, Apple, and AMD hardware alike. The standout is the multi-provider fallback chain, which turns the notorious unreliability of free cloud inference into a managed, self-healing resource: HelixLLM auto-discovers free models from 7+ cloud providers (Chutes, OpenRouter, HuggingFace, Nvidia, Cerebras, SambaNova, Together), scores them via LLMsVerifier on a 5-minute refresh, and routes through the ranked chain with automatic 429/5xx failover — always with local llama.cpp as a guaranteed last resort, so a request never simply fails for lack of an available provider.

Beyond raw inference, HelixLLM is a full application platform: a RAG knowledge pipeline (ingestion, chunking, embedding, vector search) and a ReAct agent system with tool calling, conversation sessions, and RAG integration ship in the same binary. The mode system pays off at the wire, too — in full mode all layers communicate via direct in-process Go calls with zero network overhead, while the identical binary, split across hosts, coordinates over gRPC, SSE, and Kafka. Rounding it out are Brotli/gzip content negotiation, SSE streaming that matches the OpenAI and Anthropic formats byte-for-byte, API-key and JWT auth with rate limiting, Prometheus metrics, OpenTelemetry tracing, and a large set of production-infrastructure Go submodules.

Why we built it

Teams need inference that is portable, standards-compatible, and resilient — without rewriting clients or being captive to one provider or one machine. HelixLLM was built so the same binary can run locally for development and scale to a multi-host production cluster, speaking the OpenAI and Anthropic dialects clients already use.

Why it's a game-changer

It collapses an entire inference stack — gateway, local inference, cloud fallback, RAG, and agents — into one binary controlled by a mode switch, so the architecture you deploy is a runtime decision instead of a re-platforming project. And it makes something that was previously a liability into a feature: cloud-provider reliability becomes a first-class, continuously measured concern, handled by a scored, self-healing fallback chain that reranks providers every few minutes and always degrades to guaranteed local inference.

The capability that unlocks is a single endpoint you can actually depend on — standards-compatible, portable from laptop to cluster, and incapable of going dark because one upstream provider rate-limited or failed.

What's innovative

- A single binary with a six-mode system that runs all-in-one or as distributed roles — direct in-process Go calls in full mode, gRPC/SSE/Kafka when split — so deployment topology changes without a code change or a network tax you didn't ask for.
- A scored, auto-discovering multi-provider fallback chain across 7+ free providers, continuously ranked by LLMsVerifier with automatic 429/5xx failover and a guaranteed llama.cpp last resort — free-tier capacity turned into dependable capacity.
- Dual OpenAI- *and* Anthropic-compatible surfaces served over HTTP/3 with automatic HTTP/2 fallback, so clients from either ecosystem connect without modification.
- Local inference that spans CUDA, Metal, and ROCm from one codebase — the same build runs accelerated on Nvidia, Apple, and AMD hardware.

Biggest technical challenges & how we solved them

- **Scaling from one host to many without a rewrite.** Most systems force a hard boundary between “local dev” and “distributed production,” and crossing it means re-architecting. We erased that boundary with a mode system on a single binary: the same layers talk via direct in-process calls in full mode and transparently switch to gRPC/SSE/Kafka across distributed modes, so scaling out is a configuration change rather than a port.
- **Unreliable, rate-limited free cloud providers.** Free-tier inference is fast until it 429s or disappears mid-request. We made it dependable by auto-discovering available models, scoring them with LLMsVerifier, tracking rate-limit headers proactively to route away from providers about to throttle, and failing over automatically down the ranked chain to local llama.cpp — so the pool's flakiness never reaches the caller.
- **Client compatibility across two ecosystems.** Rewriting clients to adopt a new inference backend is a non-starter. We implemented both the OpenAI *and* Anthropic API shapes — down to their distinct SSE streaming formats — so SDKs from either camp point at HelixLLM and just work.

Tech stack

- **Go + Gin** — chosen because a single-binary, concurrency-first runtime is what makes the whole mode system possible: one build that can be a laptop server or a cluster role. It carries the entire system and the gateway's HTTP layer.
- **HTTP/3 (QUIC) + TLS 1.3, with HTTP/2 fallback** — chosen for modern, low-latency, connection-resilient transport, exposed as the server surface with automatic negotiation so clients that can't do QUIC quietly drop to HTTP/2.
- **llama.cpp (CUDA/Metal/ROCm)** — chosen for portable local inference that accelerates across Nvidia, Apple, and AMD backends from one codebase; it doubles as the guaranteed last-resort provider that keeps the fallback chain from ever bottoming out.
- **LLMsVerifier** — chosen to turn “which provider is good right now” into a number; it scores and ranks the cloud fallback chain on a 5-minute refresh so routing tracks live quality, not stale assumptions.
- **Cloud providers (Chutes, OpenRouter, HuggingFace, Nvidia, Cerebras, SambaNova, Together)** — chosen to harvest free-tier capacity across many upstreams; auto-discovered and ranked into a single failover chain so no one provider is a point of failure.
- **gRPC + SSE + Kafka** — chosen as the inter-mode transports for distributed deployments: gRPC for service-to-service calls, SSE for streaming, and Kafka for decoupled event flow between roles.
- **Vector store / embeddings** — chosen to power the RAG knowledge pipeline end to end: ingest, chunk, embed, and search over documents that ground the model's answers.
- **Prometheus + OpenTelemetry** — chosen for metrics and distributed tracing that follow a request across whichever modes are deployed.
- **vasic-digital Go submodules** — chosen to reuse hardened production-infrastructure primitives instead of rebuilding them, keeping the system's foundation consistent with the wider stack.

Status & honesty notes

- **Status: beta.** Functional, actively developed distributed inference system.
- **License: TBD.** The repository declares no license in its metadata (licenseInfo null) — this is UNVERIFIED and must be resolved before stating a license.
- The canonical repository currently resolves to github.com/HelixDevelopment/llm; the HelixLLM path redirects to it.

Coverage-threshold and submodule-count figures in the README are self-reported.

Priority tier: Helix-primary.